Personalized Dynamic Knowledge-Aware Recommendation with Hybrid Explanations

Hao Sun[1], Zijian Wu[1], Yue Cui[1], Liwei Deng[1], Yan Zhao[2], and Kai Zheng[1](✉)

[1] School of Computer Science and Engineering,
University of Electronic Science and Technology of China, Chengdu, China
{wuzijian,deng_liwei}@std.uestc.edu.cn, zhengkai@uestc.edu.cn
[2] Aalborg University, Aalborg, Denmark
yanz@cs.aau.dk

Abstract. Explainable recommendation is attracting more and more attention in both industry and research communities. While some existing models utilize reviews for improving the performance of recommender systems, most of them assume that user's preference is static and each review's importance is user-independent. However, it is intuitive that user's preference is always dynamically changing and reviews from similar users should be given more importance as they share similar tastes. Moreover, they achieve model explainability at either feature level that is too concise or review level that is too redundant. To deal with these problems, we propose a **P**ersonalized **D**ynamic **K**nowledge-aware **R**ecommender (**PDKR**) for dynamic user modeling and personalized item modeling. In particular, we model user's preference with defined entities and relations in sequential knowledge graphs and capture its dynamics with a novel interval-aware Gated Recurrent Unit (GRU). Furthermore, by leveraging self-attention mechanism, we can not only learn each review's user-specific importance, but also provide tailored explanations for each user at both feature level and review level. We conduct extensive experiments on three benchmark datasets from Amazon and Yelp and the results show that PDKR outperforms all the state-of-the-art recommendation approaches in rating prediction task while providing more effective explanations simultaneously.

Keywords: Recommender system · Knowledge graph · Gated Recurrent Unit · Attention mechanism · Rating prediction

1 Introduction

Explainable recommendation has been attracting increasing attention as previous studies show that it can not only improve user's acceptance of recommended items [8,16], but also improve the transparency, persuasiveness, effectiveness, trustworthiness and satisfaction of recommender systems [1,7,17,20]. To make

© Springer Nature Switzerland AG 2021
C. S. Jensen et al. (Eds.): DASFAA 2021, LNCS 12683, pp. 148–164, 2021.
https://doi.org/10.1007/978-3-030-73200-4_10

recommender systems more explainable, user reviews, which contain rich preference information, have been widely leveraged to improve the explainability of recommendation models. For example, HFT [11] is proposed to explain the recommendation results by linking each dimension of the latent vector with a hidden topic learned from reviews. Following this idea, RBLT [15] achieves recommendation explanations by showing the topic words that have the highest recommendability score learned from a unified semantic space. NARRE [3] utilizes an attention mechanism to select the most useful reviews as the explanation. DER [5] designs a "user-aware" attention network to select the most important sentence as the explanation.

Despite their improvement in explainability, these methods suffer from some inherent limitations. To begin with, while most of them take reviews as their model input, they fail to fully exploit side information, such as user's behavior patterns and item's attributes hidden in these reviews, and thus they would lead to poor performance when the data is sparse. More importantly, most of them represent each user as a static vector and do not take user's preference dynamics into consideration. It is worth mentioning that though some RNN-based methods including DER aim to capture user's dynamic preferences by modeling each item chronologically, we believe that modeling user's preference based on each single discrete item may not be robust since a single item usually cannot reflect user's preference and would introduce occasionality into the model. Besides, these models assume that each item has a global vector suitable for all users. However, considering different users may have very different tastes, and thus they may care about different attributes of the same item, we believe that each item's vector should contain more attribute information that the target user cares about. Therefore, it is intuitive to give higher weight to the reviews of the users who are similar to the target user and obtain a user-specific item representation.

To tackle the problems mentioned above, we design a novel recommender framework based on Knowledge Graph (KG), namely **P**ersonalized **D**ynamic **K**nowledge-aware **R**ecommender (**PDKR**), for dynamic user modeling and personalized item modeling, while providing hybrid explanations at both feature level and review level. Firstly, we construct sequential knowledge graphs and extract triples that can reflect user's behavior patterns and item's attributes contained in each sequential knowledge graph. Then, we design a multi-layer graph attention network to capture user's short-term preference reflected in each sequential knowledge graph. Next, we capture user's dynamic preferences by adopting Gated Recurrent Units (GRUs) as the basic architecture to incorporate user's short-term preference and long-term preference. Though GRU is good at modeling sequential information, it does not consider the time interval information which is a very important signal since user's preference is relatively stable within a short period of time but can change dramatically when the time gap is large. To overcome this limitation, we incorporate time interval into the reset gate and the update gate to better distinguish user's historical and current preference. It is noteworthy that different from traditional RNN-based methods,

we model user's dynamic preferences based on each sequence of items rather than each single discrete item. We further construct a self-attention layer, which could automatically learn the similarity between users, in order to make item representations more personalized. Finally, we improve the explainability of the recommendation results by showing selected reviews from the most similar users (review-level explanation) with featuring words highlighted (feature-level explanation).

The main contributions of this work are summarized as follows:

- We design a novel KG-based framework that leverages rich side information to achieve dynamic user modeling and user-specific item modeling simultaneously. To the best of our knowledge, it is the first attempt to combine knowledge graph and GRUs to capture user's dynamic preferences and address the data sparsity problem in rating prediction tasks.
- We modify the original GRU architecture by introducing time interval to make it aware of the user's preferences evolving with time. We also leverage a self-attention mechanism to learn the similarity between users and obtain user-specific item representation. By combining review-level and feature-level explanations, we can improve the effectiveness and satisfaction of the recommendation.
- We conduct extensive experiments on three benchmark datasets to evaluate the performance of our model. The favorable results verify our expectation that the proposed framework can reach a high prediction accuracy and provide more effective explanation at the same time.

2 Related Work

2.1 Explainable Recommendation

Explainable recommendation models have proved to be quite useful in ranges of applications. Many techniques have been proposed to improve the explainability of recommendation models. Early works [4,11,21] mainly focus on improving the explainability of matrix factorization models and collaborative filtering models while preserving their accuracy. For example, EFM [21] extends Matrix Factorization by aligning latent dimensions with explicit features and provides feature-level explanations. However, feature-level explanations can only provide very limited information and sometimes lead to ambiguity without sufficient contextual information. Recently, with the advance of deep learning, deep models [3,5,14] have been proposed to improve the accuracy and explainability of recommendation simultaneously. Basically, in these models, side information like reviews is utilized to enhance the explainability. NARRE [3], D-Attn [14] and DER [5] automatically learn the importance of different sentences and select the most important sentences as the review-level explanation of the recommendation. Although review-level explanation can provide more detailed description of items, sometimes they contain too much redundant information which is useless and distracting, thus affecting the quality of explanations.

To address the problems mentioned above, our model provides review-level explanations while highlighting the featuring words with their sentiment polarity suggested in corresponding colors. Therefore, users can choose to look at the featuring words only or refer to the review context when they cannot fully understand the featuring words.

2.2 Knowledge Graph Based Recommendation

Knowledge Graph (KG) based recommendation is attracting more and more attention since KG contains rich information about users and items, which can help enhance recommendation performance and address the problem of data sparsity. In general, existing KG-based recommendation studies can be divided into two categories. The first category is embedding-based methods. For example, Collaborative Knowledge based Embedding (CKE) [19] combines collaborative filtering with auxiliary knowledge embeddings such as text embedding and image embedding. The second category is path-based methods which leverage path information for recommendation. For example, Personalized Entity Recommendation (PER) [18] treats each knowledge graph as a heterogeneous information network and extracts meta-path-based latent features to represent the connectivity between users and items.

Although these models achieve promising results, most of them do not take user's preference dynamics into consideration. To address this issue, we aim to model user's dynamic preferences by chronologically organizing knowledge graphs as a sequence and utilizing a GRU architecture to incorporate user's historical and current preference.

3 Preliminaries

To formulate the problem, we first introduce the definitions of interaction sequence and sequential knowledge graph.

Definition 1 (Interaction Sequence). *Each interaction* $o = (u, v, r, w, t)$ *includes user ID* u*, item ID* v*, rating* r*, review* w *and time* t*, which denotes that user* u *interacts with item* v *at time* t *by rating* r *and review* w*. Given a historical interaction set* $O = [o_1, o_2, .., o_{|O|}]$ *of user* u *and sequence length* λ*, an interaction sequence* $s_i = [o_{i,1}, o_{i,2}, ..., o_{i,\lambda}]$ *is a chronologically-ordered continuous subset of* O*. The set of interaction sequences of user* u *is represented as* $S_u = [s_1, s_2, ..., s_m]$*, where* m *is the number of interaction sequences.*

Definition 2 (Sequential Knowledge Graph). *Given an interaction sequence* s_i*, a sequential knowledge graph denoted as* $\mathcal{G}_i = (\mathcal{E}, \mathcal{R})$ *can be constructed by extracting defined entities and relations from* s_i*, where* $\mathcal{E}$ *is the set of entities and* $\mathcal{R}$ *is the set of relations. All sequential knowledge graphs constructed from* S_u *is represented as* $G_u = \{\mathcal{G}_1, \mathcal{G}_2, ..., \mathcal{G}_m\}$*.*

Problem Statement. Given a set of users U and their historical interactions in the training set D_{tr}, we aim to accurately predict the rating r that a user u will assign to item v by learning the embeddings of users and items, and provide effective explanation at both review level and feature level.

Sequential Knowledge Graph Construction. This paper aims to leverage reviews for product recommendation, which usually provides three kinds of entities including user entity, item entity and feature entity. To capture user behavior patterns and item attributes, we convert the interaction between these entities into the following four groups of triplet facts, which are illustrated in Fig. 1.

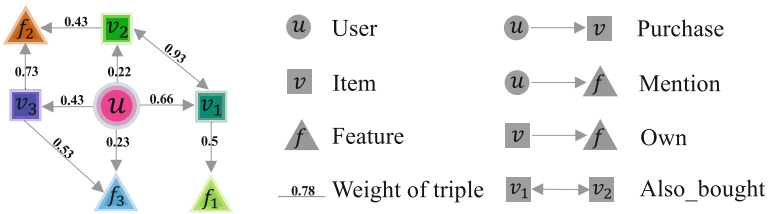


Fig. 1. One example of the sequential knowledge graph

(1) $(u, Purchase, v)$ represents the interaction that user u bought item v. The weight of the relation is defined as the rating that user u assigned to item v.
(2) (v, Own, f) represents the interaction that item v owns feature f. The weight of the relation is defined as the sentiment polarity that the user expressed on the feature, where 1 means the user likes the feature while 0.5 means the user dislikes the feature[1].
(3) $(u, Mention, f)$ represents the interaction that user u mentioned feature f more than once in her review history. The weight of the relation is defined as the frequency that user u mentioned feature f.
(4) $(v_1, AlsoBought, v_2)$ represents the interaction that item v_1 and item v_2 were bought together in one transaction more than once. The weight of the relation is defined as the frequency that the two items appeared in the same transaction.

An advantage of defining these relations is that user's behavior patterns and item attributes can be incorporated into a sequential knowledge graph in a unified manner. More specifically, by defining relation $Mention$ and $AlsoBought$, user's special preferences on some features and item's interdependence with each other can be captured. Besides, the strength of relations can be reflected by the weights of them. Note that other kinds of entities (such as categories and

[1] The sentiment polarity value should have been 1 (positive) or -1 (negative). We modify the negative value -1 to 0.5, which can be seen as how well the item performs on the feature.

brands) and their corresponding relations can also be added to capture specific user behavior patterns and item attributes.

All the weights are normalized within their relation types. With the entities and relations defined above, we can now construct sequential knowledge graphs by extracting triples (e_h, r, e_t) from each interaction sequence s_i, where e_h represents the head entity, e_t represents the tail entity and r represents the relation between them.

4 The Proposed Model

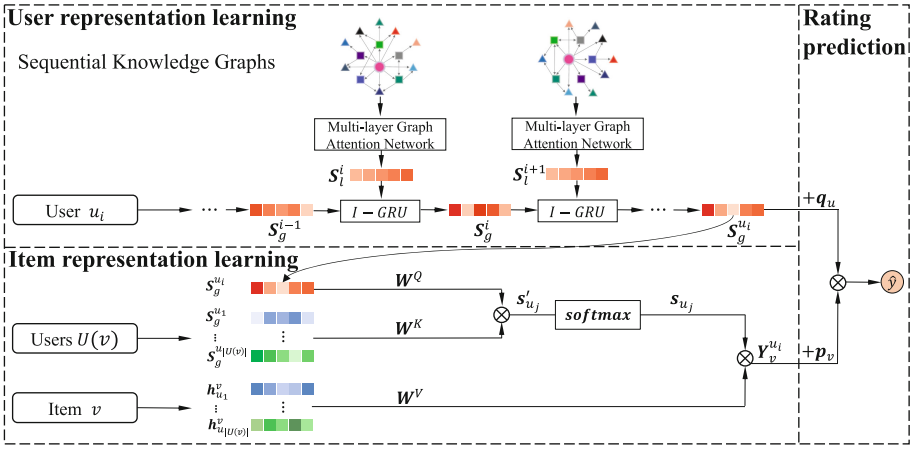


Fig. 2. Framework overview

As shown in Fig. 2, our proposed Personalized Dynamic Knowledge-aware Recommender (PDKR) consists of three parts: user representation learning, item representation learning and rating prediction. In the user representation learning part, we first design a multi-layer graph attention network to learn user's local sequence representation which represents user's short-term preference reflected in the current session. Then, we re-design the architecture of GRU to incorporate user's long-term preference reflected in previous sessions and user's short-term preference reflected in the current session, and obtain user's global sequence representation eventually. In the item representation learning part, we leverage a self-attention layer for user-specific item representation learning. Finally, we combine user representation and item representation to obtain the predicted rating in the rating prediction part.

4.1 Local Sequence Representation Learning

As illustrated in Fig. 3, given user u's sequential knowledge graphs $G_u = \{\mathcal{G}_1, \mathcal{G}_2, ..., \mathcal{G}_m\}$, we first update the embeddings of entities and relations, and

then aggregate all the entity embeddings in the sequential knowledge graph $\mathcal{G}_i$ to obtain user u's local sequence representation S_l^i which represents user u's short-term preference reflected in the interaction sequence s_i.

Entity Embedding Update. To distinguish user entity from other kinds of entities, we first transform user entity embeddings and other kinds of entity embeddings into different latent space before the graph attention layer by applying the following equation:

$$\boldsymbol{h}_i^{user/else} = \boldsymbol{W}_1^{user/else}\boldsymbol{x}_i, \tag{1}$$

where $\boldsymbol{x}_i \in \mathbb{R}^d$ is the embedding of entity e_i, $\boldsymbol{W}_1^{user}, \boldsymbol{W}_1^{else} \in \mathbb{R}^{d\times d}$ are the transformation matrices of user entity and other kinds of entities respectively.

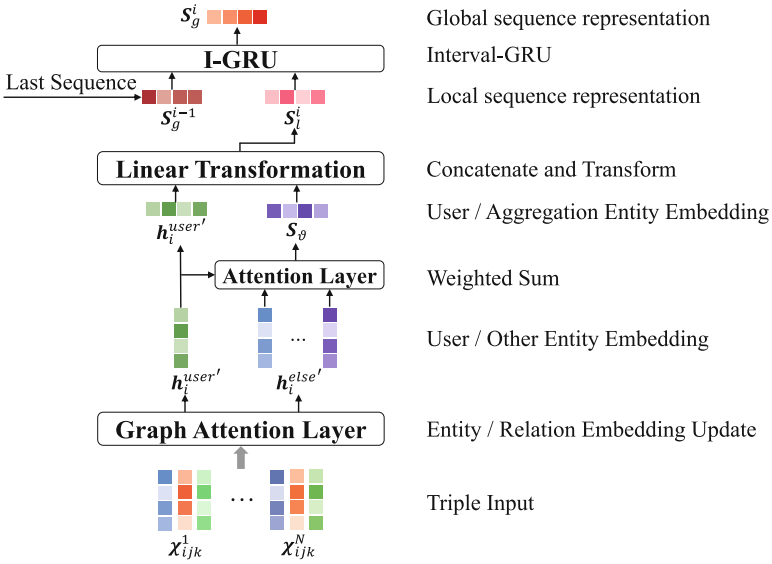


Fig. 3. Structure of user representation learning

In graph attention layer, we first calculate each triple's relative attention value, and then we concatenate the outputs of M independent attention layers where in each layer we calculate the weighted average of triple embedding separately:

$$\begin{aligned}
\boldsymbol{\chi}_{ijk} &= \boldsymbol{W}_2[\boldsymbol{h}_i \| \boldsymbol{h}_j \| \boldsymbol{g}_k] \\
c_{ijk} &= w_{ijk} LeakyReLU(\boldsymbol{W}_3 \boldsymbol{\chi}_{ijk}) \\
\alpha_{ijk} = softmax(c_{ijk}) &= \frac{exp(c_{ijk})}{\sum_{n\in N_i}\sum_{r\in R_{in}} exp(c_{inr})} \\
\boldsymbol{h}_i^{'} &= \|_{m=1}^{M} \sigma(\sum_{j\in N_i} \alpha_{ijk}^m \boldsymbol{\chi}_{ijk}^m),
\end{aligned} \tag{2}$$

where $\boldsymbol{h}_i$ and $\boldsymbol{h}_j$ denote embeddings of entities e_i and e_j respectively, $\boldsymbol{\chi}_{ijk}$ denotes the triple (e_i, r_k, e_j), w_{ijk} denotes the weight of the relation in the triple (e_i, r_k, e_j), N_i denotes the neighbors of entity e_i and R_{in} denotes the set of relations between entity e_i and entity e_n. As a common practice of multi-head attention models, we employ averaging instead of concatenating in the final layer to get the final embedding of entities.

Relation Embedding Update. Similar to [13], linear transformation is then performed on the relation embedding matrix $\boldsymbol{G}$ to get the final relation embedding.

$$\boldsymbol{G}^{'} = \boldsymbol{G}\boldsymbol{W}_R \tag{3}$$

Generating Local Sequence Representation. Considering that different entities in the sequential knowledge graph should have different levels of priority and hence different weights, we further adopt the soft-attention mechanism to better model user's preference reflected in her interacted entities.

$$\begin{aligned} \beta_i &= \boldsymbol{q}^T \sigma(\boldsymbol{W}_4 \boldsymbol{h}^{user'} + \boldsymbol{W}_5 \boldsymbol{h}_i^{else'} + \boldsymbol{\omega}) \\ \boldsymbol{S}_\vartheta &= \sum_{i=1}^{n} \beta_i \boldsymbol{h}_i^{else'}, \end{aligned} \tag{4}$$

where $\boldsymbol{q} \in \mathbb{R}^d$, $\boldsymbol{h}_i^{else'}$ is the updated embedding of entities except for user entity, $\boldsymbol{h}^{user'}$ is the updated embedding of user entity and n is the number of all entities involved in the sequential knowledge graph except for user entity.

Finally, we compute the local sequence representation of user u by taking linear transformation over the concatenation of user embedding and the aggregation of other entity embeddings $\boldsymbol{S}_\vartheta$[2]:

$$\boldsymbol{S}_l = \boldsymbol{W}_6[\boldsymbol{h}^{user'} \| \boldsymbol{S}_\vartheta] \tag{5}$$

Sequential Knowledge Graph Learning. To better learn entity and relation embeddings, we employ TransE [2], which learns embeddings such that for a given valid triple (e_i, r_k, e_j), the condition $\boldsymbol{h}_i + \boldsymbol{g}_k \approx \boldsymbol{h}_j$ holds. In particular, we try to learn entity and relation embeddings to minimize the L1-norm dissimilarity measure given by $g(i,k,j) = \|\boldsymbol{h}_i + \boldsymbol{g}_k - \boldsymbol{h}_j\|_1$. The loss function of sequential knowledge graph learning is defined as follows:

$$\mathcal{L}_{KG} = \sum_{(i,k,j,j^{'}) \in \mathcal{T}} max\{g(i,k,j) - g(i,k,j^{'}) + \gamma, 0\}, \tag{6}$$

where $\mathcal{T} = \{(i,k,j,j^{'}) | (i,k,j) \in \mathcal{G} \ \wedge \ (i,k,j^{'}) \notin \mathcal{G}\}$ and $(i,k,j^{'})$ is a negative triple (that does not exist in the sequential knowledge graph) constructed by replacing one entity in a valid triple randomly and $\gamma > 0$ is a margin hyper-parameter.

[2] For simplicity, the label for discriminating different users and sequences is omitted.

4.2 Global Sequence Representation Learning

Intuitively, a user's preference is relatively stable within a short period of time but could change dramatically after a long time. Therefore, it is necessary to take the time interval into consideration while modeling user's dynamic preferences. To prevent the model from being too complicated, we adopt GRU [6] as the basic architecture. However, it does not take the time interval information into consideration. So we re-design the architecture of GRU by simply incorporating time interval into the reset gate $\boldsymbol{r}_t$ and the update gate $\boldsymbol{z}_t$. The newly designed GRU (called I-GRU) can learn when to infuse more historical information with its gate mechanism. The overall architecture of I-GRU is shown in Fig. 4.

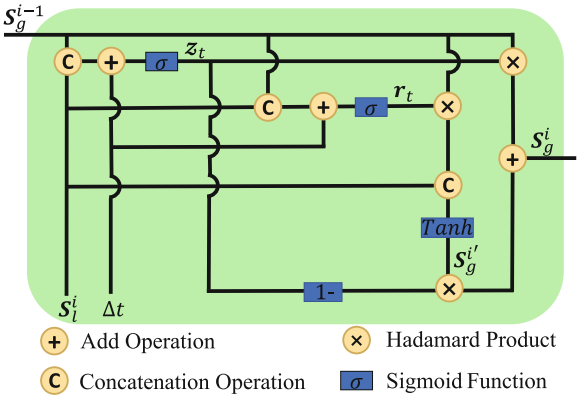


Fig. 4. The architecture of the new-designed I-GRU

Given user's previous global sequence representation $\boldsymbol{S}_g^{i-1}$ and current local sequence representation $\boldsymbol{S}_l^i$, the current global sequence representation $\boldsymbol{S}_g^i$ can be calculated as follows:

$$\begin{aligned} \boldsymbol{z}_t &= \sigma(\boldsymbol{W}_z[\boldsymbol{S}_l^i, \boldsymbol{S}_g^{i-1}] - \Delta t \boldsymbol{w}_t) \\ \boldsymbol{r}_t &= \sigma(\boldsymbol{W}_r[\boldsymbol{S}_l^i, \boldsymbol{S}_g^{i-1}] - \Delta t \boldsymbol{w}_t) \\ \boldsymbol{S}_g^{i'} &= Tanh(\boldsymbol{W}_s[\boldsymbol{S}_l^i, \boldsymbol{r}_t \odot \boldsymbol{S}_g^{i-1}]) \\ \boldsymbol{S}_g^i &= \boldsymbol{z}_t \odot \boldsymbol{S}_g^{i-1} + (1 - \boldsymbol{z}_t) \odot \boldsymbol{S}_g^{i'}, \end{aligned} \tag{7}$$

where $\boldsymbol{w}_t \in \mathbb{R}^d$, $\Delta t = t_i - t_{i-1}$ and t_i simply averages the interaction time of the sequence s_i. Therefore, when the time interval Δt is larger, which leads to smaller $\boldsymbol{z}_t$ and $\boldsymbol{r}_t$, the influence of historical information $\boldsymbol{S}_g^{i-1}$ (long-term preference) is lowered by multiplying $\boldsymbol{z}_t$ and $\boldsymbol{r}_t$, and the current input $\boldsymbol{S}_l^i$ (short-term preference) becomes the predominant factor, and vice versa.

4.3 Item Representation Learning

Traditional item modeling methods assume that each item has a global representation suitable for all users. However, considering different users may focus on different features for the same item, we believe that each item should have a user-specific representation for each user. Besides, it is intuitive that if user u_j is similar to target user u_i, then user u_j's review should be paid more attention when recommending items to target user u_i. To seamlessly integrate this observation into our model, we design a novel item representation learner adapted from self-attention, which could automatically identify the importance of different users' reviews.

To begin with, for each target user u_i, a query vector is defined as the linear transformation of the global sequence representation of u_i, which is denoted as $\boldsymbol{S}_g^{u_i}$.

$$\boldsymbol{Q}_{u_i} = \boldsymbol{W}^Q \boldsymbol{S}_g^{u_i} \tag{8}$$

Given an item v, we can get the set of users $U(v)$ who have reviews on item v. For each user $u_j \in U(v)$, we can calculate her key vector as follows:

$$\boldsymbol{K}_{u_j} = \boldsymbol{W}^K \boldsymbol{S}_g^{u_j} \tag{9}$$

Similarly, for each user $u_j \in U(v)$, we can calculate her value vector by performing a linear transformation on her entity embedding of item v.

$$\boldsymbol{V}_{u_j} = \boldsymbol{W}^V \boldsymbol{h}_{u_j}^v \tag{10}$$

Obtaining the query vector of the target user u_i and the key vector of user u_j, we can get the absolute attention value of u_j as follows:

$$s_{u_j}^{'} = \boldsymbol{K}_{u_j}^T \boldsymbol{Q}_{u_i} \tag{11}$$

Softmax is applied on $s_{u_j}^{'}$ to get the relative attention value of each user u_j, which can be seen as the similarity between target user u_i and user u_j. It is worth mentioning that the review of the user who is the most similar to the target user u_i will be selected as the review-level explanation, which will be discussed later in the explainability study section.

$$s_{u_j} = softmax(s_{u_j}^{'}) = \frac{exp(s_{u_j}^{'})}{\sum_{u_n \in U(v)} exp(s_{u_n}^{'})} \tag{12}$$

To encapsulate more information contained in the item reviews, M independent attention layers are applied to calculate the item embedding, followed by an averaging layer to obtain the final user-specific item representation.

$$\boldsymbol{Y}_v^{u_i} = \sigma(\frac{1}{M} \sum_{m=1}^{M} s_{u_j}^m \boldsymbol{V}_{u_j}^m) \tag{13}$$

4.4 Prediction

Latent Factor Model (LFM) [9] aims to predict the rating by multiplying user's representation vector and item's representation vector based on matrix factorization. We extend it by introducing auxiliary embeddings $\boldsymbol{q}_u$ and $\boldsymbol{p}_v$ for more robust prediction. Specifically, the predicted rating is calculated as :

$$\widehat{R}_{u,v} = \boldsymbol{W}_7^T(\boldsymbol{q}_u + \boldsymbol{S}_g^u) \odot (\boldsymbol{p}_v + \boldsymbol{Y}_v^u) + \boldsymbol{b}_u + \boldsymbol{b}_v + \boldsymbol{\mu}, \tag{14}$$

where $\boldsymbol{S}_g^u$ and $\boldsymbol{Y}_v^u$ are user's global sequence representation vector and item's representation vector respectively, $\boldsymbol{b}_u$, $\boldsymbol{b}_v$ and $\boldsymbol{\mu}$ denote user bias, item bias and global bias respectively.

4.5 Training

Since the goal of this paper is rating prediction, which is actually a regression problem, we adopt Root Mean Square Error (RMSE) as the loss function:

$$\mathcal{L}_{rec} = \sum_{(u,v)\in\mathcal{D}_{tr}} (\widehat{R}_{u,v} - R_{u,v})^2, \tag{15}$$

where $\mathcal{D}_{tr}$ denotes the set of user-item pairs in the training set.

Finally, we have the objective function to optimize Eq. 6 and 15 jointly as follows:

$$\mathcal{L}_{PDKR} = \mathcal{L}_{rec} + \mathcal{L}_{KG} + \xi\|\Theta\|_2^2, \tag{16}$$

where Θ is the set of model parameters to be regularized.

5 Experiments

5.1 Datasets

In our experiments, we use three publicly available real-world datasets in different domains to evaluate our models and baselines. Two datasets are from Amazon[3]: **Movies_and_TV** and **Cell_Phones_and_Accessories**, which contain user reviews and ratings of products in movie entertainment category and electronic product category respectively. Another dataset is **Yelp**[4] which is a large-scale dataset consisting of restaurant reviews and ratings. Amongst them, Yelp is the largest dataset that contains more than 2.1 million reviews while Cell_Phones_and_Accessories is the smallest dataset which contains about 106 thousand reviews. These datasets are selected to cover different domains and scales, which can demonstrate the robustness of our model. Besides, we adopt the state-of-the-art approach described in [10,22] to extract features from reviews.

For each user interaction sequence, we divide them into 6:2:2 for training, validation and testing respectively. To ensure the KG quality, we filter out users and items with less than 20 ratings. The characteristics of our datasets are shown in Table 1.

[3] http://deepyeti.ucsd.edu/jianmo/amazon/.
[4] https://www.kaggle.com/yelp-dataset/yelp-dataset/data.

Table 1. Basic statistics of evaluation datasets

Datasets	#user	#items	#features	#reviews
Phones	4696	35011	697	106148
Movies	30161	41245	1987	1235945
Yelp	47944	58870	3660	2148994

5.2 Baselines and Implementation Details

To evaluate the performance of our proposed PDKR model, five state-of-the-art methods are selected as baselines:

- **PMF** [12] is a traditional matrix factorization method that uses Gaussian distribution to model the latent factors for users and items.
- **HFT** [11] is a traditional topic modeling based method that projects user's latent vector into the latent topic space with Latent Dirichlet Allocation (LDA).
- **DeepCoNN** [23] is a deep recommendation model that jointly models user behaviors and item properties using textual reviews. Authors have shown that it outperformed other topic modeling based methods.
- **NARRE** [3] is a state-of-the-art explainable recommendation method that first introduces a novel attention mechanism to explore the usefulness of reviews.
- **DER** [5] is a state-of-the-art explainable method that introduces time-aware GRU to model user's dynamic preferences and provide review-level explanation.

We implement our PDKR model using Pytorch. In our model, the batch size is fixed at 64, the learning rate is initialized as 5×10^{-5}, the margin hyper-parameter γ is initialized as 1 and the weight decay parameter ξ is initialized as 1×10^{-5}. The embedding size d is chosen in the range of {50, 100, 150, 200} and the sequence length λ is chosen in the range of {3,5,7,9,11}. All the algorithms are implemented on an Intel(R) Xeon(R) CPU E5-2650 v4 @ 2.20 GHz and four GeForce GTX 2080 GPUs.

5.3 Evaluation Metric

To evaluate the performance of all methods, we select RMSE as the evaluation metric, which is widely used for rating prediction in recommender systems. Given a predicted rating $\widehat{R}_{u,v}$ and a ground-truth rating $R_{u,v}$ from user u to item v, the RMSE score is calculated as:

$$RMSE = \sqrt{\frac{1}{|\mathcal{D}_{ts}|} \sum_{(u,v)\in\mathcal{D}_{ts}} (\widehat{R}_{u,v} - R_{u,v})^2}, \tag{17}$$

where $\mathcal{D}_{ts}$ is the set of user-item pairs in the testing set.

5.4 Evaluation on Rating Prediction

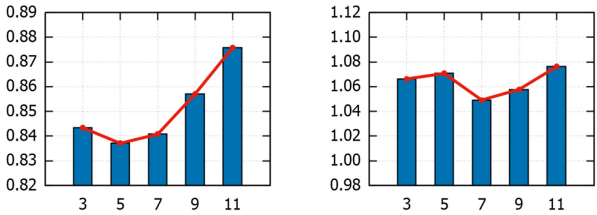
Table 2. Performances for all methods in terms of RMSE

Dataset	Phones	Movies	Yelp
PMF	1.2164	1.2980	1.3716
HFT	1.0525	1.1528	1.2401
DeepCoNN	0.9546	1.0486	1.1637
NARRE	0.9365	1.0323	1.1571
DER	0.9278	1.0331	1.1404
PDKR	**0.8371**	**0.9879**	**1.0492**
Improvement of PDKR	9.8%	4.4%	8.0%

Overall Performance Comparison. From the results shown in Table 2, we can see that: the simple PMF method performs the worst in all methods because it is a traditional collaborative filtering model which only takes the rating information into consideration and it fails to capture user's dynamic preferences. While HFT, DeepCoNN and NARRE all take review information as the model input, NARRE performs the best. This is because NARRE leverages a novel attention mechanism to identify more useful reviews, hence the impact of valuable reviews can be enhanced while the influence of noise from useless reviews can be reduced. DER performs better than NARRE because it not only takes the usefulness of reviews into account, but also leverages T-GRU to capture user's dynamic preferences. Our model PDKR achieves 9.8%, 4.4%, 8.0% relative improvement on Cell_Phones_and_Accessories, Movies_and_TV and Yelp respectively over DER, which demonstrates the superiority of our model. In our model, we not only fully exploit review information with the defined entities and relations in sequential knowledge graphs, but also propose a simple but powerful GRU variant to capture user's dynamic preferences. Moreover, by leveraging self-attention mechanism, we can obtain more user-specific item representation for each item. Therefore, the latent representations for both users and items are more accurate and eventually lead to higher accuracy in rating prediction.

Effect of Sequence Length. Sequence length λ is an important hyperparameter that directly influences how much information can be incorporated into a sequential knowledge graph and eventually influences RMSE. Due to space limit, we only report the results on Cell_Phones_and_Accessories dataset and Yelp dataset.

The sequence length is tuned in the range of {3,5,7,9,11}, and 5,7 is chosen to be the default value of λ on Cell_Phones_and_Accessories dataset and Yelp dataset respectively. From the results shown in Fig. 5, we can observe that both too high sequence length and too low sequence length would affect the model performance negatively. To explain, when the number of entities in a sequential knowledge graph is too large, it is difficult to find a suitable representation of



(a) On Phones (b) On Yelp

Fig. 5. Effect of sequence length

the whole graph which contains too much information. On the other hand, when few entities are used, the representation of a sequential knowledge graph cannot reflect user's preference, which would lead to poor performance.

Effect of Data Sparsity. To simulate the effect of data sparsity, We randomly reduce the training data by a ratio of 5%, 10%, 15% and 20%.

Table 3. Effect of data sparsity on Cell_Phones_and_Accessories and Yelp

Mask	Model					
	Cell_Phones_and_Accesories					
	PMF	HFT	DeepConn	NARRE	DER	PDKR
0	1.2164	1.0525	0.9546	0.9365	0.9278	**0.8371**
5	1.2179	1.0748	0.9756	0.9464	0.9428	**0.8451**
10	1.2189	1.1037	0.9849	0.9541	0.9618	**0.8752**
15	1.2255	1.1102	1.0035	0.9827	0.9939	**0.8852**
20	1.2266	1.1583	1.0165	1.0105	1.0331	**0.9095**
Mask	Model					
	Yelp					
	PMF	HFT	DeepConn	NARRE	DER	PDKR
0	1.3716	1.2401	1.1637	1.1571	1.1404	**1.0492**
5	1.3809	1.2679	1.1715	1.1654	1.1491	**1.0548**
10	1.4121	1.2833	1.1817	1.1838	1.1664	**1.0627**
15	1.4247	1.3020	1.2081	1.1944	1.1792	**1.0802**
20	1.4592	1.3038	1.2313	1.2009	1.1799	**1.0928**

As shown in Table 3, when the data is sparser, all methods' accuracies decrease. However, PDKR still performs the best among all methods, which indicates the robustness of our model. One possible explanation is that, with the use of sequential knowledge graphs, we can explore more valuable information from user's reviews, which can remedy the problem of data sparsity to some extent.

5.5 Explainability Study

Previous works on explainable recommendation mainly focus on providing either feature-level or review-level explanations. However, both of them suffer from some inherent limitations. For feature-level explanations, simply providing a feature the user is interested in is too concise and often causes confusion without contextual information, leading to low user satisfaction. For review-level explanations, it is too redundant to take the whole review as the explanation since reviews are highly subjective and often contain many useless personal feelings, leading to low effectiveness. In this paper, we combine review-level explanations and feature-level explanations. In particular, we select the most important reviews of (dis)recommended item v for target user u_i, and highlight the featuring words and corresponding sentiment words in the reviews.

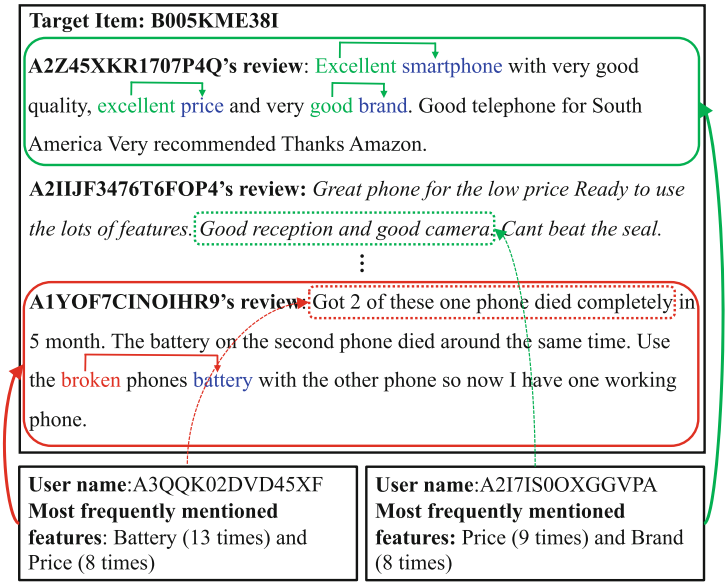


Fig. 6. The example of explanations generated for two different users on the same item. The most important review learned by NARRE is labeled by italic. The reviews from the most similar users selected by PDKR are presented in full line boxes while the sentences selected by DER are presented in dotted line boxes. Green and red suggest the explanation for why the target item is recommended and disrecommended respectively. The target user and corresponding explanation are linked by a directed arrow. (Color figure online)

To showcase the superiority of our model's explainability, we present an example in Fig. 6. Specifically, we compare the explanations of a target item generated by NARRE, DER and our model PDKR for different users. As we

can see, NARRE highlights the same review for different users due to its user-independent attention mechanism while our model can provide personalized review information, which is more practical and reasonable in real scenarios. In the explanation generated by PDKR, featuring words are highlighted in blue while opinion words are highlighted in green when the sentiment is positive and in red otherwise. It should be noted that highlighting the featuring words gives users more options. For example, when the featuring word is clear enough as shown in user A2Z45XKR1707P4Q's review, user can choose to ignore the redundant body of the review. However, when the featuring word is confusing as shown in user A1YOF7CINOIHR9's review, user can refer to the context of the review to better understand why the battery is described by the word "broken", and thus both effectiveness and satisfaction of the recommendation can be improved. Though DER selects different sentences for different users, it does not exhibit such capability due to its lack of contextual information and would sometimes lead to confusion as shown in the explanation it generated for target user A3QQK02DVD45XF in the red dotted line box.

6 Conclusion

In this paper, we propose a novel knowledge-aware framework named PDKR, which models user's dynamic preferences and explores reviews' user-specific usefulness simultaneously in the context of explainable recommendation. Extensive experiments have been conducted on three large real-world datasets from Amazon and Yelp. The proposed PDKR outperforms all the state-of-the-art recommendation models in rating prediction task. Explainability study shows that we can provide personalized explanations for target user on each (dis)recommended item, which improves both effectiveness and satisfaction of the recommendation.

As for future work, we will investigate the potential of incorporating visual information into knowledge graphs for better recommendation performance and more comprehensive recommendation explanations.

Acknowledgements. This work is supported by NSFC (No. 61972069, 61836007, 61832017) and Sichuan Science and Technology Program under Grant 2020JDTD0007.

References

1. Bilgic, M., Mooney, R.J.: Explaining recommendations: satisfaction vs. promotion. In: Beyond Personalization Workshop, IUI, vol. 5, p. 153 (2005)
2. Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J., Yakhnenko, O.: Translating embeddings for modeling multi-relational data. In: NIPS, pp. 2787–2795 (2013)
3. Chen, C., Zhang, M., Liu, Y., Ma, S.: Neural attentional rating regression with review-level explanations. In: WWW, pp. 1583–1592 (2018)
4. Chen, X., Qin, Z., Zhang, Y., Xu, T.: Learning to rank features for recommendation over multiple categories. In: SIGIR, pp. 305–314 (2016)
5. Chen, X., Zhang, Y., Qin, Z.: Dynamic explainable recommendation based on neural attentive models. In: AAAI, vol. 33, pp. 53–60 (2019)

6. Cho, K., et al.: Learning phrase representations using RNN encoder-decoder for statistical machine translation. arXiv preprint arXiv:1406.1078 (2014)
7. Cramer, H., et al.: The effects of transparency on trust in and acceptance of a content-based art recommender. User Model. User-Adap. Inter. **18**(5), 455 (2008). https://doi.org/10.1007/s11257-008-9051-3
8. Herlocker, J.L., Konstan, J.A., Riedl, J.: Explaining collaborative filtering recommendations. In: CSCW, pp. 241–250 (2000)
9. Koren, Y., Bell, R., Volinsky, C.: Matrix factorization techniques for recommender systems. Computer **42**(8), 30–37 (2009)
10. Lu, Y., Castellanos, M., Dayal, U., Zhai, C.: Automatic construction of a context-aware sentiment lexicon: an optimization approach. In: WWW, pp. 347–356 (2011)
11. McAuley, J., Leskovec, J.: Hidden factors and hidden topics: understanding rating dimensions with review text. In: RecSys, pp. 165–172 (2013)
12. Mnih, A., Salakhutdinov, R.R.: Probabilistic matrix factorization. In: NIPS, pp. 1257–1264 (2008)
13. Nathani, D., Chauhan, J., Sharma, C., Kaul, M.: Learning attention-based embeddings for relation prediction in knowledge graphs. arXiv preprint arXiv:1906.01195 (2019)
14. Seo, S., Huang, J., Yang, H., Liu, Y.: Interpretable convolutional neural networks with dual local and global attention for review rating prediction. In: RecSys, pp. 297–305 (2017)
15. Tan, Y., Zhang, M., Liu, Y., Ma, S.: Rating-boosted latent topics: understanding users and items with ratings and reviews. In: IJCAI, vol. 16, pp. 2640–2646 (2016)
16. Tintarev, N., Masthoff, J.: A survey of explanations in recommender systems. In: ICDE, pp. 801–810 (2007)
17. Tintarev, N., Masthoff, J.: Designing and evaluating explanations for recommender systems. In: Ricci, F., Rokach, L., Shapira, B., Kantor, P.B. (eds.) Recommender Systems Handbook, pp. 479–510. Springer, Boston, MA (2011). https://doi.org/10.1007/978-0-387-85820-3_15
18. Yu, X., et al.: Personalized entity recommendation: a heterogeneous information network approach. In: WSDM, pp. 283–292 (2014)
19. Zhang, F., Yuan, N.J., Lian, D., Xie, X., Ma, W.Y.: Collaborative knowledge base embedding for recommender systems. In: SIGKDD, pp. 353–362 (2016)
20. Zhang, Y., Chen, X.: Explainable recommendation: a survey and new perspectives. arXiv preprint arXiv:1804.11192 (2018)
21. Zhang, Y., Lai, G., Zhang, M., Zhang, Y., Liu, Y., Ma, S.: Explicit factor models for explainable recommendation based on phrase-level sentiment analysis. In: SIGIR, pp. 83–92 (2014)
22. Zhang, Y., Zhang, H., Zhang, M., Liu, Y., Ma, S.: Do users rate or review? Boost phrase-level sentiment labeling with review-level sentiment classification. In: SIGIR, pp. 1027–1030 (2014)
23. Zheng, L., Noroozi, V., Yu, P.S.: Joint deep modeling of users and items using reviews for recommendation. In: WSDM, pp. 425–434 (2017)